Semana 2 · ESP32, wolfSSL 5.9.4 y simulador

Fase de la propuesta: revisión y validación con el cliente (semanas 1-2).

Objetivo	Instalar el entorno del ESP32 (ESP-IDF v5.5 + wolfSSL 5.9.4), medir el costo de ML-KEM en el ESP32 y probar toda la pila TLS en la Raspberry con el simulador, antes de cablear.
Dedicación estimada	6–8 horas (la instalación de ESP-IDF tarda).
Antes de empezar	Semana 1 cerrada. ESP32 y cable USB de datos. Laptop con internet.
Al terminar tendrán	Firmware del proyecto compilando, benchmark de wolfSSL en el ESP32 y la prueba simulada aprobada en la Raspberry.

Paso 1. Instalar ESP-IDF v5.5 en la laptop

Windows

- Descarguen el instalador oficial de Espressif (ESP-IDF Installation Manager / ESP-IDF Tools Installer) desde la documentación de ESP-IDF y elijan la versión **v5.5**.
- Al terminar, abran el acceso directo **«ESP-IDF 5.5 PowerShell»**. Todos los comandos `idf.py` se ejecutan ahí.

Linux o macOS

```
mkdir -p ~/esp && cd ~/esp
git clone -b v5.5 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf && ./install.sh esp32
. ./export.sh                # repetir en cada terminal nueva
idf.py --version             # debe decir v5.5
```

Paso 2. Conectar el ESP32 y probar «hello world»

Conecten el ESP32 a la laptop. Identifiquen el puerto: en Windows, **Administrador de dispositivos → Puertos** (p. ej. COM5; si no aparece, instalen el driver CP210x o CH340 según el chip USB de su placa). En Linux, `/dev/ttyUSB0`.

```
cp -r $IDF_PATH/examples/get-started/hello_world ~/esp/
cd ~/esp/hello_world
idf.py set-target esp32
idf.py -p PUERTO build flash monitor          # PUERTO = COM5 o /dev/ttyUSB0; salir: Ctrl+]
```

Anoten en la bitácora lo que imprime: modelo de chip, número de núcleos, revisión y tamaño de flash.

Paso 3. Descargar wolfSSL 5.9.4 y definir WOLFSSL_ROOT

```
git clone --depth 1 --branch v5.9.4-stable https://github.com/wolfSSL/wolfssl.git ~/esp/wolfssl
# Linux/macOS (agregar al ~/.bashrc para que sea permanente):
export WOLFSSL_ROOT=~/esp/wolfssl
# Windows (PowerShell de ESP-IDF):
#   setx WOLFSSL_ROOT "C:\Users\<usuario>\esp\wolfssl"     y abran una ventana nueva
```

Paso 4. Medir ML-KEM y X25519 en el ESP32 (benchmark de wolfSSL)

Este dato es valioso para el capítulo de resultados: cuánto tarda el ESP32 en las operaciones criptográficas por sí solas, sin el bus.

```
cd ~/esp/wolfssl/IDE/Espressif/ESP-IDF/examples/wolfssl_benchmark
idf.py set-target esp32
idf.py menuconfig
#   Component config -> wolfSSL -> wolfSSL Post Quantum Kyber KEM PQ -> [*] Enable ML-KEM
#   Guardar (S) y salir (Q)
idf.py -p PUERTO build flash monitor
```

Copien toda la salida del benchmark en `docs/evidencias/benchmark_wolfssl_esp32.txt`. Busquen las líneas de ML-KEM 768 (generación de claves, encapsulación, desencapsulación) y de X25519.

> **Si falla**
>
> Si CMake dice que no encuentra wolfSSL, la variable `WOLFSSL_ROOT` no está definida en esa terminal. Si ML-KEM no aparece en la salida, anótenlo y sigan: no bloquea el proyecto.

Paso 5. Compilar el firmware del proyecto

```
git clone https://github.com/kali2010-x/proyecto-kyber-rpi.git ~/esp/proyecto
cd ~/esp/proyecto
python herramientas/cert_a_header.py          # crea esp32/firmware/main/cert_servidor.h
cd esp32/firmware
idf.py set-target esp32
idf.py build
idf.py size                                   # anoten el tamaño de la aplicación
```

> **sdkconfig.defaults solo se aplica si no existe sdkconfig**
>
> Si alguna vez compilaron antes y cambian `sdkconfig.defaults`, borren el archivo `sdkconfig` y ejecuten `idf.py fullclean` antes de compilar.
>
> Comprueben las opciones clave: `grep -E "SLAVE_DRIVER_VERSION_2|MLKEM|FREERTOS_HZ" sdkconfig` debe mostrar las tres activadas (=y, =y, =1000).

> **Errores típicos al compilar**
>
> - `i2c_slave_write` no declarado: falta `CONFIG_I2C_ENABLE_SLAVE_DRIVER_VERSION_2=y` (ver recuadro anterior).
> - `cert_servidor.h: No such file`: no ejecutaron `cert_a_header.py` (necesita `certs/servidor.pem`; hagan `git pull`).
> - Cualquier otro error: copien las últimas 30 líneas y me las pasan.

Paso 6. Compilar wolfSSL 5.9.4 en la Raspberry (para el simulador)

El simulador corre en la Raspberry el **mismo código C** del ESP32. Permite probar el servidor TLS y la capa de enlace sin hardware y, más adelante, separar problemas de software de problemas de cableado.

```
cd ~ && git clone --depth 1 --branch v5.9.4-stable https://github.com/wolfSSL/wolfssl.git
cd wolfssl && ./autogen.sh
./configure --enable-tls13 --enable-mlkem --enable-curve25519 --enable-opensslextra \
            --disable-examples --prefix=$HOME/wolfssl-inst
make -j4 && make install                      # 5-15 minutos
cd ~/proyecto-kyber-rpi/sim && make WOLF=$HOME/wolfssl-inst
```

Paso 7. Prueba completa en simulación

Usen dos ventanas de tmux (`tmux`; nueva ventana: Ctrl+b c; cambiar: Ctrl+b n).

```
# Ventana 1: el "ESP32" simulado
cd ~/proyecto-kyber-rpi/sim && ./esclavo_sim ../certs/servidor.pem
#   debe imprimir: wolfSSL 5.9.4 (se requiere >= 5.9.4), FRAME_SIZE=128

# Ventana 2: el servidor
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
export OPENSSL_CONF=~/proyecto-kyber-rpi/pi/conf/openssl_grupos.cnf
python3 servidor_tls.py --sim --grupo clasico | tee ../docs/evidencias/sim_clasico.txt
python3 servidor_tls.py --sim --grupo hibrido | tee ../docs/evidencias/sim_hibrido.txt
python3 ataque.py --sim
```

Esperado: en las dos corridas `"error": ""`, `"esp_estado": 0`, `"cierre_ok": true` y `"grupo_detectado"` igual al grupo pedido; en `ataque.py`: `TODAS LAS PRUEBAS PASARON`.

Agreguen OPENSSL_CONF al venv para no olvidarlo

```
echo 'export OPENSSL_CONF=~/proyecto-kyber-rpi/pi/conf/openssl_grupos.cnf' \
     >> ~/proyecto-kyber-rpi/src/venv/bin/activate
```

Criterio de cierre de la semana

- ☐ ESP-IDF v5.5 instalado; hello_world funciona y anotaron los datos del chip.
- ☐ WOLFSSL_ROOT apunta a wolfSSL 5.9.4.
- ☐ benchmark_wolfssl_esp32.txt guardado en docs/evidencias/.
- ☐ El firmware del proyecto compila (idf.py build sin errores) y anotaron su tamaño.
- ☐ En la Raspberry, el simulador muestra wolfSSL 5.9.4 y ambas corridas simuladas terminan sin error.
- ☐ ataque.py --sim dice TODAS LAS PRUEBAS PASARON.
- ☐ Commit y push de evidencias y bitácora.